

RESPONSI PRAKTIKUM
STRUKTUR DATA DAN ALGORITMA
2026

IDENTITAS

Nama : Reyhan Diandra Alvaro

NIM : L0125029

Kelas : A

Judul Program : Sistem Music Player Berbasis Struktur Data Java

Deskripsi Program : Program ini merupakan aplikasi music player sederhana berbasis Java yang menerapkan berbagai struktur data seperti List, Stack, Queue, Set, Map, Tree, dan Binary Search Tree. Program memiliki fitur menambahkan lagu, memutar lagu, membuat playlist, menampilkan riwayat pemutaran, mencari lagu berdasarkan judul dan genre, memberikan rekomendasi lagu, serta menampilkan lagu terpopuler berdasarkan jumlah pemutaran.

Dokumentasi Program

Analisis Kode

1.

```
// class data lagu
static class lagu {
    String judul; // judul lagu
    String genre; // genre lagu
    int puter = 0; // jumlah lagu diputar

    // constructor lagu
    lagu(String j, String g) {
        judul = j;
        genre = g;
    }

    // nambah jumlah play
    void putar() {
        puter++;
    }
}
```

Kode ini digunakan untuk membuat sebuah lagu, di mana class berfungsi sebagai template untuk menyimpan data judul, genre, dan puter (jumlah lagu diputar). Di bawahnya terdapat konstruktor lagu untuk memasukkan komponen-komponen lagu secara langsung saat objek dibuat. Terakhir, terdapat fungsi nambah jumlah play yang apabila dipanggil akan secara otomatis meningkatkan jumlah puter.

Analisis Kompleksitas: Operasi pembuatan objek dan penambahan jumlah putar ini berjalan sangat instan. Karena tidak ada proses looping atau pencarian, kompleksitas waktunya adalah konstan, yaitu $O(1)$.

2.

```
// struktur data

static List<lagu> listlagu = new ArrayList<>(); // list semua lagu
static Stack<lagu> history = new Stack<>(); // stack history lagu
static Queue<lagu> antrian = new LinkedList<>(); // queue playlist
static Set<String> noduplikat = new HashSet<>(); // set anti duplikat
static Map<String, lagu> carilagu = new HashMap<>(); // map search cepat
static Map<String, List<lagu>> genretree = new HashMap<>(); // genre -> list lagu
```

Ini adalah bagian buat menyiapkan wadah-wadah data lagu pake berbagai jenis struktur. Ada listlagu buat nampung semua lagu, history (Stack) untuk menyimpan riwayat lagu terbaru di tumpukan atas, dan antrian (Queue) buat playlist biar muternya berurutan. Agar menjaga nggak

ada lagu ganda dipake, noduplikat (Set), carilagu (Map) buat nyari judul lagu secara cepat, dan genretree buat ngelompokin lagu berdasarkan genre.

Analisis Kompleksitas: Memasukkan dan mengambil data dari Stack dan Queue membutuhkan waktu $O(1)$. Begitu juga dengan Set dan Map yang menggunakan sistem hashing, proses pengecekan duplikat dan pencarian datanya berjalan sangat efisien dengan kompleksitas waktu rata-rata $O(1)$.

3.

```
static class genrenode {  
  
    // nama genre  
    String genre;  
  
    // list lagu dalam genre  
    List<lagu> daftarlagu = new ArrayList<>();  
  
    // anak node genre  
    List<genrenode> anak = new ArrayList<>();  
  
    genrenode(String g) {  
        genre = g;  
    }  
}  
  
// root tree genre  
static genrenode rootgenre = new genrenode("music");
```

Bagian ini fungsinya membuat struktur genre yang modelnya kayak pohon bercabang (Tree). Di dalam satu node genre ini, ada nama genrenya dan sebuah daftar buat nampung lagu-lagu yang punya genre sama. Ada juga bagian anak buat nyimpen cabang-cabang genre lain di bawahnya. Terakhir, ada rootgenre dengan nama "music" yang fungsinya sebagai pusat tempat semua kategori genre bermuara.

Analisis Kompleksitas: Pembuatan kerangka atau node node untuk setiap genre baru berjalan sangat cepat dengan kompleksitas waktu $O(1)$

4.

```
// node bst  
static class node {  
  
    // node nyimpen object lagu  
    lagu data;  
  
    // hubungan ke node lain  
    node kiri, kanan;  
  
    // constructor node  
    node(lagu l) {  
        // lagu yg dikirim disimpan ke node  
        data = l;  
    }  
}  
  
// root bst  
static node root;
```

Ini bagian buat bikin node buat pohon BST (Binary Search Tree) kita. Di dalam node ini isinya ada data lagu yang mau disimpan, serta punya "tangan" kiri dan kanan buat nyambungin ke node lagu lainnya. Di dalamnya juga ada konstruktor agar saat node baru dibuat datanya otomatis masuk, dan terakhir ada root yang fungsinya sebagai titik awal atau "akar" utama tempat kita mulai nyusun urutan lagu.

Analisis Kompleksitas: Mengisi data lagu ke dalam struktur node baru hanya memerlukan satu operasi penugasan (assignment), sehingga kompleksitasnya adalah $O(1)$.

5.

```
// insert bst diurut berdasarkan play count
static node insert(node now, lagu baru) {

    // kalau kosong bikin node baru
    if (now == null)
        return new node(baru);

    // play lebih kecil ke kiri
    if (baru.puter < now.data.puter)
        now.kiri = insert(now.kiri, baru);

    // selain itu ke kanan
    else
        now.kanan = insert(now.kanan, baru);

    return now;
}
```

Fungsi insert ini tugasnya mengatur cara lagu masuk ke dalam pohon BST berdasarkan jumlah putarnya. Aturannya: kalau tempatnya masih kosong, dia bakal bikin node baru; kalau sudah ada isinya, dia bakal bandingin kalau jumlah putar lagu baru lebih kecil, diarahkan ke kiri, dan kalau lebih besar atau sama, ditaruh ke sebelah kanan.

Analisis Kompleksitas: Proses insert pada Binary Search Tree memiliki kompleksitas waktu rata-rata $O(\log N)$ karena proses pencarian posisi data dilakukan dengan menelusuri cabang pohon. Namun, pada kondisi terburuk ketika pohon tidak seimbang, kompleksitas dapat meningkat menjadi $O(N)$.

6.

```
// masukin lagu ke tree genre
static void tambahgenre(lagu baru) {

    // cek genre sudah ada atau belum
    for (genrenode g : rootgenre.anak) {
        // kalau genre sama
        if (g.genre.equals(baru.genre)) {
            g.daftarlagu.add(baru); // masukin lagu ke genre itu
            return;
        }
    }

    genrenode barugenre = new genrenode(baru.genre); // kalau genre belum ada bikin genre baru
    barugenre.daftarlagu.add(baru); // masukin lagu pertama
    rootgenre.anak.add(barugenre); // sambung ke root
}
```

Fungsi ini tugasnya untuk memasukan lagu baru ke dalam struktur pohon (Tree) genrenya. Caranya, program bakal ngecek dulu satu satu ke daftar anak genre yang sudah nempel di pusat atau root. Kalau genrenya ketemu yang cocok, lagu itu langsung dimasukan ke dalam kelompok daftar lagu di genre tersebut. Tapi, kalau setelah dicek ternyata genrenya belum ada, fungsi ini bakal otomatis bikin node genre baru, masukin lagu pertamanya ke situ, terus langsung nyambungkan node kategori baru itu ke pusat(rootgenre).

Analisis Kompleksitas: Karena program harus melakukan *looping* untuk mencocokkan nama genre pada cabang anak, maka kompleksitas waktunya adalah $O(N)$, di mana N adalah jumlah kategori genre yang sudah terbentuk saat itu.

7.

```
// tambah lagu baru
static void tambahlagu(String judul, String genre) {

    // cek duplikat
    if (noduplikat.contains(judul)) {
        System.out.println("lagu sudah ada!");
        return;
    }

    lagu baru = new lagu(judul, genre); // bikin object lagu bar

    listlagu.add(baru); // masuk list
    noduplikat.add(judul); // masuk set
    carilagu.put(judul, baru); // masuk map

    // kalau genre belum ada bikin list baru
    genretree.putIfAbsent(genre, new ArrayList<>());
    genretree.get(genre).add(baru); // masukin lagu ke genre

    tambahgenre(baru); // masuk tree genre
    root = insert(root, baru); // masuk bst

    System.out.println("tambah lagu: " + judul);
}
```

Fungsi ini tugasnya buat "muter" lagu yang kita pilih dari daftar. Pas dijalankan, jumlah putar lagu itu bakal otomatis nambah satu, terus lagunya langsung dimasukan ke wadah history (Stack) biar posisi lagu yang paling baru diputar ada di tumpukan paling atas. setiap kali lagu diputar, fungsi ini bakal ngerombak dan nyusun ulang seluruh pohon BST dari nol; tujuannya biar urutan lagu paling populer selalu update dan akurat karena ada lagu yang baru aja nambah jumlah putarnya.

Analisis Kompleksitas: Proses pemutaran lagu melibatkan penambahan data ke Stack, pembaruan genre, serta penyusunan ulang Binary Search Tree. Karena program melakukan perulangan terhadap data lagu, kompleksitas waktu keseluruhan secara rata-rata adalah $O(N \log N)$.

8.

```
// tampil semua lagu
static void tampilsemualagu() {
    int i = 1;

    // loop semua lagu
    for (lagu l : listlagu) {

        System.out.println(i++ + ". " + l.judul + " | " + l.genre + " | Played: " + l.putar);
    }
}
```

Fungsi ini tugasnya buat nampilin semua daftar lagu yang sudah tersimpan di dalam aplikasi kita. Caranya, program bakal melakukan looping atau keliling ke seluruh isi wadah listlagu (ArrayList) satu per satu dari urutan pertama sampai terakhir. Setiap kali ketemu satu data lagu, program bakal langsung nyetak keterangan lengkapnya ke layar, mulai dari nomor urut, judul, genre, sampai jumlah berapa kali lagu itu sudah diputar biar kita bisa lihat semua koleksi yang ada secara berurutan.

Analisis Kompleksitas: Karena program harus ngunjungi setiap lagu satu per satu supaya bisa dicetak semuanya tanpa ada yang kelewat, maka kompleksitas waktunya adalah $O(N)$, di mana N adalah jumlah total lagu yang tersimpan. Artinya, kalau koleksi lagu lo makin banyak, waktu yang dibutuhkan buat nampilin daftar ini bakal bertambah secara linear atau berbanding lurus dengan banyaknya jumlah lagu tersebut.

9.

```
// muter lagu
static void putarlagu(lagu l) {
    l.putar(); // play count naik
    history.push(l); // masuk history

    // bst dibikin ulang karena play count berubah
    root = null;

    // masukan ulang semua lagu
    for (lagu x : listlagu) {
        root = insert(root, x);
    }

    System.out.println("\nnow playing: " + l.judul);
}
```

Fungsi ini tugasnya buat "muter" lagu yang kita pilih. Pas dipanggil, jumlah putar lagu itu langsung nambah satu dan lagunya dimasukin ke wadah history (Stack) biar terekam sebagai lagu terakhir yang diputar. Yang unik, fungsi ini bakal ngereset dan ngebangun ulang seluruh pohon BST dari awal pakai semua lagu yang ada di listlagu, tujuannya supaya urutan kepopuleran lagu di dalam pohon tetap update dan akurat gara-gara ada lagu yang jumlah putarnya baru aja berubah.

Analisis Kompleksitas: Mengubah data dan masuk ke Stack butuh waktu $O(1)$. Namun, karena program harus melakukan *looping* ke semua lagu (sebanyak N) dan memasukkannya satu per satu ke BST (butuh $O(\log N)$ tiap masuk), maka proses bongkar-pasang pohon ini membuat kompleksitas waktunya menjadi $O(N \log N)$.

10.

```
// pilih lagu dari index
static void putarbyindex(Scanner sc) {

    tampilsemualagu();

    System.out.print("pilih lagu: ");

    int idx = sc.nextInt();
    sc.nextLine();

    // cek index valid
    if (idx >= 1 && idx <= listlagu.size()) {
        putarlagu(listlagu.get(idx - 1));
    } else {
        System.out.println("index salah!");
    }
}
```

Fungsi ini tugasnya buat memudahkan kita memilih lagu yang mau diputar berdasarkan nomor urut (indeks) yang ada di daftar. Pertama, dia bakal manggil fungsi buat nampilin semua lagu biar kita bisa lihat nomornya, terus program bakal nunggu kita masukin angka pilihan. Kalau angka yang kita masukin bener (nggak lebih kecil dari 1 atau nggak ngelewati jumlah total lagu), maka lagu tersebut bakal langsung diputar lewat fungsi putarlagu. Tapi kalau angkanya tidak ada, program cuma bakal ngasih tahu kalau pilihannya salah.

Analisis Kompleksitas: Mengambil data pakai indeks di List itu instan $O(1)$. Menampilkan lagu butuh $O(N)$. Tapi, karena di akhirnya dia memanggil fungsi putarlagu yang harus

ngebangun ulang pohon, maka kompleksitas waktu paling lambat yang mendominasi fungsi ini tetap $O(N \log N)$.

11.

```
// bikin playlist input angka dipisah spasi
static void bikinplaylist(Scanner sc) {
    tampilsemualagu();
    System.out.print("input nomor lagu: ");

    String input = sc.nextLine();
    String[] arr = input.split(" ");

    boolean valid = true;

    // loop semua input
    for (String x : arr) {
        try {
            int idx = Integer.parseInt(x);
            // cek nomor lagu valid
            if (idx >= 1 && idx <= listlagu.size()) {
                antrian.offer(listlagu.get(idx - 1));
            }
            // kalau nomor gak ada
            else {
                valid = false;
                break;
            }
        }
    }
}
```

```
// kalau input bukan angka
catch (Exception e) {
    valid = false;
    break;
}
// hasil akhir
if (valid)
    System.out.println("playlist dibuat!");
else {
    // kalau ada input salah
    antrian.clear();

    System.out.println("input gak ada!");
}
```

Fungsi ini bertugas untuk menyusun daftar putar (playlist) dengan menerima input berupa deretan nomor lagu yang dipisahkan oleh spasi. Program akan memecah string input tersebut menjadi potongan angka, lalu memvalidasi setiap nomor satu per satu. Disini ada sistem pembatalan otomatis: jika ada input yang tidak valid langsung mengosongkan kembali wadah antrian (Queue) dan membatalkan seluruh pembuatan playlist untuk memastikan data yang masuk benar-benar akurat.

Analisis Kompleksitas Waktu: Karena fungsi ini memanggil tampilsemualagu di awal, prosesnya memerlukan waktu $O(N)$. Selanjutnya, program melakukan perulangan sebanyak jumlah angka yang diinput (M). Di dalam perulangan tersebut, proses validasi dan memasukkan lagu ke Queue bersifat instan atau $O(1)$. Jika terjadi kesalahan input, fungsi `antrian.clear()` juga berjalan sangat cepat. Dengan demikian, total kompleksitas waktu untuk fungsi ini adalah $O(N + M)$.

12.

```
// muter playlist pake queue
static void putarplaylist() {
    // cek playlist kosong
    if (antrian.isEmpty()) {
        System.out.println("playlist kosong!");
        return;
    }

    // loop sampai playlist habis
    while (!antrian.isEmpty()) {
        // ambil depan queue
        lagu l = antrian.poll();

        // putar lagu
        putarlagu(l);
    }
}
```


fungsi ini tugasnya buat muterin semua lagu yang sudah kita masukan ke daftar antrian (playlist) sampai habis. Program bakal ngecek dulu, kalau antriannya kosong ya dia nggak bakal muter apa-apa. Tapi kalau ada isinya, program bakal pakai sistem First-In-First-Out (FIFO), di mana dia bakal ngambil lagu paling depan dari wadah antrian (Queue), muterin lagunya pakai fungsi putarlagu, terus lanjut ke lagu berikutnya sampai semua isi antrian itu kosong.

Analisis Kompleksitas : Karena fungsi ini pakai perulangan while buat ngabisin isi antrian, maka kompleksitasnya tergantung dari jumlah lagu yang ada di playlist tersebut, yaitu $O(M)$ (di mana M adalah jumlah lagu dalam antrian). Namun, karena di dalam setiap perulangan dia manggil fungsi putarlagu yang harus ngebangun ulang pohon BST ($O(N \log N)$), maka secara keseluruhan proses muter playlist ini bakal jadi yang paling berat, yaitu $O(M * N \log N)$. Makin panjang playlist yang lo buat, makin sering programnya kerja keras buat nge-update urutan kepopuleran lagu di setiap putarannya.

13.

```
// tampil history lagu
static void tampilhistory() {

    // cek history kosong
    if (history.isEmpty()) {
        System.out.println("belum ada history!");
        return;
    }

    System.out.println("\nhistory lagu");

    // tampil dari atas stack
    for (int i = history.size() - 1; i >= 0; i--) {
        System.out.println("- " + history.get(i).judul);
    }
}
```

Fungsi ini tugasnya buat nampilin daftar lagu apa saja yang baru-baru ini sudah diputar oleh pengguna. Karena menggunakan wadah history (Stack), program bakal ngecek dulu apakah ada isinya atau tidak. Kalau ada, program bakal ngebaca data dari urutan paling atas (yang paling terakhir diputar) sampai ke bawah, sehingga user bisa melihat riwayat musik mereka secara terbalik dari yang paling baru ke yang paling lama.

Analisis Kompleksitas : Untuk mengecek apakah riwayat kosong dan mengambil ukuran data, waktunya sangat cepat yaitu $O(1)$. Namun, karena program harus melakukan perulangan (looping) untuk mencetak setiap judul lagu yang ada di dalam riwayat satu per satu, maka

kompleksitas waktunya adalah $O(N)$, di mana N adalah jumlah lagu yang ada di dalam history tersebut. Semakin banyak lagu yang telah diputar oleh pengguna, maka semakin banyak pula data riwayat yang harus ditampilkan oleh program.

14.

```
// cari lagu pake map search o(1)
static void carilagu(String judul) {
    lagu l = carilagu.get(judul); // ambil lagu dari map

    // kalau gak ada
    if (l == null) {
        System.out.println("lagu gak ketemu!");
        return;
    }
    System.out.println("\nlagu ketemu"); // print data lagu
    System.out.println(l.judul+ " | "+ l.genre+ " | Played: " + l.puter);
}
```

Fungsi ini tugasnya buat nyari lagu secara instan cuma dengan mengetikkan judulnya saja. Berbeda dengan cara manual yang harus ngecek lagu satu-satu dari awal sampai akhir, fungsi ini langsung "menunjuk" ke lokasi lagu tersebut di dalam wadah carilagu (HashMap). Kalau judulnya ada di database, data lengkap lagu itu bakal langsung muncul; tapi kalau nggak ada, program bakal langsung ngasih tahu kalau lagunya nggak ketemu.

Analisis Kompleksitas: Ini adalah bagian paling efisien dalam program kita karena menggunakan struktur data Map yang punya kompleksitas waktu rata-rata $O(1)$. Baik jumlah lagu sedikit maupun sangat banyak, proses pencarian tetap berjalan cepat karena struktur data HashMap memungkinkan akses data secara langsung tanpa melakukan perulangan terhadap seluruh data lagu.

15.

```
// lagu paling populer node paling kanan bst
static void topsong() {

    // cek bst kosong
    if (root == null) {
        System.out.println("belum ada lagu!");
        return;
    }
    node temp = root;

    // jalan terus ke kanan karena kanan lebih besar
    while (temp.kanan != null) {
        temp = temp.kanan;
    }
    System.out.println("\ntop song");// tampil lagu terbesar
    System.out.println(temp.data.judul+ " | Played: "+ temp.data.puter);
}
```

Fungsi ini tugasnya buat nyari lagu mana yang paling sering diputar atau paling populer di aplikasi kamu. Karena kita pakai struktur BST (Binary Search Tree) di mana lagu dengan jumlah putar lebih besar selalu ditaruh di sebelah kanan, Program akan menelusuri node paling kanan pada Binary Search Tree karena lagu dengan jumlah pemutaran terbesar disimpan pada cabang kanan pohon.

Analisis Kompleksitas: Pada kondisi pohon BST yang seimbang, proses pencarian lagu paling populer dapat berjalan lebih cepat karena program hanya menelusuri sebagian cabang pohon saja. Namun, pada kondisi terburuk ketika pohon sangat miring ke satu sisi, kompleksitas waktunya dapat mencapai $O(N)$.

16.

```
// rekomendasi lagu genre sama
static void rekomendasi() {

    // cek history kosong
    if (history.isEmpty()) {
        System.out.println("belum ada lagu diputar!");
        return;
    }
    lagu terakhir = history.peek(); // ambil lagu terakhir diputar
    System.out.println("\nrekomendasi");

    List<lagu> rec = genretree.get(terakhir.genre); // ambil list genre sama

    // loop semua lagu genre sama
    for (lagu l : rec) {
        // jangan tampil lagu yg sama
        if (!l.judul.equals(terakhir.judul)) {
            System.out.println("- " + l.judul);
        }
    }
}
```

Fungsi ini tugasnya buat kasih saran lagu lain yang genrenya mirip dengan lagu yang baru saja kamu putar. Caranya, program bakal mengecek lagu paling atas di history (Stack) pakai fungsi peek, terus dia bakal nyari node genre yang sama di dalam genretree (Map). Setelah ketemu daftar lagunya, program bakal nampilin semua lagu yang ada di kategori genre tersebut, tapi dia cukup pintar buat nggak nampilin lagi judul lagu yang baru aja kamu dengerin tadi.

Analisis Kompleksitas: Untuk ngambil lagu terakhir dari Stack dan nyari kategori genre di Map, waktunya sangat instan yaitu $O(1)$. Namun, karena program harus ngelakuin perulangan

(looping) buat nampilin semua lagu yang ada di dalam kategori genre tersebut, maka kompleksitas waktunya adalah $O(N)$,

17.

```
// cari lagu berdasarkan genre pake tree
static void carigenre(Scanner sc) {
    System.out.print("input genre: ");
    String cari = sc.nextLine();

    // loop semua genre di tree
    for (genreNode g : rootgenre.anak) {

        // kalau genre ketemu
        if (g.genre.equalsIgnoreCase(cari)) {

            System.out.println("\nGenre: " + g.genre);
            // tampil semua lagu genre itu
            for (lagu l : g.daftarlagu) {
                System.out.println("- " + l.judul+ " | Played: "+ l.puter);
            }
            return;
        }
    }
    System.out.println("genre gak ada!");
}
```

Fungsi ini tugasnya buat nyari dan nampilin semua daftar lagu berdasarkan kategori genre yang kita ketik. Program bakal menelusuri struktur Tree dengan cara keliling ke setiap cabang anak (child node) yang nempel di pusat atau rootgenre. Kalau nama genre yang kita cari ketemu, program bakal ngebuka node genre tersebut dan nyetak semua daftar lagu yang tersimpan di dalamnya satu per satu.

Analisis Kompleksitas: Karena program harus melakukan perulangan (looping) untuk mengecek nama genre di setiap cabang anak, maka proses pencariannya memakan waktu $O(N)$, di mana N adalah jumlah kategori genre yang ada. Setelah genrenya ketemu, program melakukan looping lagi buat nyetak semua lagu di genre itu, sehingga kompleksitas totalnya adalah $O(N + G)$, dengan G adalah jumlah lagu dalam genre tersebut. Karena konstanta tidak dihitung maka kompleksitasnya adalah $O(N)$.

18.

```
public static void main(String[] args) {  
  
    Scanner sc = new Scanner(System.in);  
  
    // lagu default  
    tambahlagu("Yellow", "Pop");  
    tambahlagu("Fix You", "Pop");  
    tambahlagu("Numb", "Rock");  
    tambahlagu("Believer", "Rock");  
    tambahlagu("Perfect", "Pop");  
    tambahlagu("i love you so much", "Pop");  
    tambahlagu("GOD", "R&B");  
    tambahlagu("take me home", "Country");  
}
```

program bakal otomatis masukan beberapa lagu bawaan (default) biar aplikasinya nggak kosong saat pertama kali dibuka, terus dia bakal nampilin daftar pilihan menu di layar. Program ini bakal terus standby dalam perulangan (looping) buat nungguin perintah dari user, dan baru bakal berhenti kalau user milih opsi buat keluar dari aplikasi.

```
while (true) {  
  
    System.out.println("\n===== MUSIC PLAYER =====");  
  
    System.out.println("1. tambah lagu");  
    System.out.println("2. tampil lagu");  
    System.out.println("3. putar lagu");  
    System.out.println("4. bikin playlist");  
    System.out.println("5. putar playlist");  
    System.out.println("6. history");  
    System.out.println("7. cari lagu");  
    System.out.println("8. top song");  
    System.out.println("9. rekomendasi");  
    System.out.println("10. cari genre");  
    System.out.println("0. keluar");  
  
    System.out.print("pilih: ");  
    System.out.println(" ");  
  
    int pilih = sc.nextInt();  
    sc.nextLine();  
}
```

Bagian ini merupakan main method yang berfungsi menjalankan program, menambahkan lagu default, serta menampilkan menu utama secara berulang hingga pengguna memilih keluar dari aplikasi.

19.

```
switch (pilih) {  
    case 1:  
        System.out.print("judul: ");  
        String j = sc.nextLine();  
  
        System.out.print("genre: ");  
        String g = sc.nextLine();  
  
        tambahlagu(j, g);  
        break;  
  
    case 2:  
        tampilsemualagu();  
        break;  
  
    case 3:  
        putarbyindex(sc);  
        break;  
  
    case 4:  
        bikinplaylist(sc);  
        break;  
  
    case 5:  
        putarplaylist();  
        break;  
  
    case 6:  
        tampilhistory();  
        break;  
  
    case 7:  
        System.out.print("cari judul: ");  
        String s = sc.nextLine();  
  
        carilagu(s);  
        break;  
  
    case 8:  
        topsong();  
        break;  
  
    case 9:  
        rekomendasi();  
        break;  
  
    case 10:  
        carigenre(sc);  
        break;  
  
    case 0:  
        return;  
  
    default:  
        System.out.println("pilihan gak ada!");  
}
```

Struktur switch case digunakan untuk mengarahkan pilihan menu pengguna ke fungsi yang sesuai pada program.

Analisis Kompleksitas: Kompleksitas bergantung pada fungsi yang dipanggil di setiap case.
Analisis Program (saat dijalankan)

1.

```
tambah lagu: Yellow
tambah lagu: Fix You
tambah lagu: Numb
tambah lagu: Believer
tambah lagu: Perfect
tambah lagu: i love you so much
tambah lagu: GOD
tambah lagu: take me home

===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
```

Ini adalah awal ketika program dijalankan dimana ada tampilan lagu default yang ditambahkan. Lalu akan muncul UI dari music player dimana ada 11 pilihan yang bisa digunakan oleh user.

2.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
1
judul: To the bone
genre: sad
tambah lagu: To the bone
```

lalu apabila user memilih 1, atau tambah lagu maka user diminta memasukan judul dan genre lagunya dan lagu akan otomatis tersimpan.

3.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
2
1. Yellow | Pop | Played: 0
2. Fix You | Pop | Played: 0
3. Numb | Rock | Played: 0
4. Believer | Rock | Played: 0
5. Perfect | Pop | Played: 0
6. i love you so much | Pop | Played: 0
7. GOD | R&B | Played: 0
8. take me home | Country | Played: 0
9. To the bone | sad | Played: 0
```

Lalu apabila kita memilih opsi 2 atau tampilkan lagu maka akan muncul lagu yang sudah tersimpan (lagu default + lagu yang kita tambahkan). Dan di opsi lagu akan terlihat juga judul,genre, dan jumlah diputarnya.

4.

```
pilih:
3
1. Yellow | Pop | Played: 0
2. Fix You | Pop | Played: 0
3. Numb | Rock | Played: 0
4. Believer | Rock | Played: 0
5. Perfect | Pop | Played: 0
6. i love you so much | Pop | Played: 0
7. GOD | R&B | Played: 0
8. take me home | Country | Played: 0
9. To the bone | sad | Played: 0
pilih lagu: 2

now playing: Fix You
```

Apabila kita memilih opsi 3 yaitu play lagu akan muncul list lagu dan kita disuruh memasukan nomer lagu yang akan kita putar sebagai contoh jika kita memilih 2 maka lagu Fix you akan otomatis terputar dan keluar notifnya.

5.

```
pilih:
4
1. Yellow | Pop | Played: 0
2. Fix You | Pop | Played: 1
3. Numb | Rock | Played: 0
4. Believer | Rock | Played: 0
5. Perfect | Pop | Played: 0
6. i love you so much | Pop | Played: 0
7. GOD | R&B | Played: 0
8. take me home | Country | Played: 0
9. To the bone | sad | Played: 0
input nomor lagu: 1 2 3 5 8
playlist dibuat!

===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
5

now playing: Yellow

now playing: Fix You

now playing: Numb

now playing: Perfect

now playing: take me home
```

Lalu saat kita memilih 4 atau membuat playlist maka akan muncul list lagu dan kita disuruh memasukan lagu mana saja yang akan dibuat playlist. Dan jika sudah maka playlist akan otomatis tersimpan.

Setelah itu jika kita memilih 5 atau putar playlist maka playlist yang sudah kita buat tadi akan di putar.

```
pilih:
4
1. Yellow | Pop | Played: 0
2. Fix You | Pop | Played: 1
3. Numb | Rock | Played: 0
4. Believer | Rock | Played: 0
5. Perfect | Pop | Played: 0
6. i love you so much | Pop | Played: 0
7. GOD | R&B | Played: 0
8. take me home | Country | Played: 0
9. To the bone | sad | Played: 0
input nomor lagu: 1 3 5 6 7 6
input gak ada!
```

Sebagai sistem anti eror apabila kita memasukan nomer lagu playlist yang tidak ada maka akan muncul peringatan input nggak ada dan playlist batal dibuat.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
5
playlist kosong!
```

Lalu jika kita memutar playlist yang belum dimasukan atau masih kosong maka akan muncul playlist kosong.

6.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
6

history lagu
- take me home
- Perfect
- Numb
- Fix You
- Yellow
- Fix You
```

Lalu sekarang menu histori atau nomer 6 maka akan memunculkan history lagu yang terakhir kita putar maka akan menjadi paling atas karena disini sistemnya menggunakan Stack

7.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
7
cari judul: Yellow

lagu ketemu
Yellow | Pop | Played: 1
```

sekarang ke menu 7 yaitu cari lagu disini sistemnya menggunakan map jadi kita hanya perlu memasukan judul lagu maka map akan mencari seluruh detail tentang lagu tersebut.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
7
cari judul: ysysy
lagu gak ketemu!
```

apabila lagunya tidak ada atau salah maka akan keluar peringatan yaitu Lagu nggak ketemu.

8.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
8

top song
Fix You | Played: 2
```

Ini adalah opsi ke 8 yaitu top song disini user akan mendapatkan lagu yang paling sering kita putar. Sistem dari ini adalah kita mencari lagu yang paling banyak kita play dengan tree

9.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
9

rekomendasi
```

Lalu ada menu rekomendasi. Sistem kerja dari rekomendasi ini adalah dia bekerja dengan memanfaatkan apa yang terakhir kita putar contoh Rock maka akan keluar lagu lagu yang bergenre rock, pada case ini karena kita belum memutar lagu maka rekomendasinya kosong.

```

pilih:
3
1. Yellow | Pop | Played: 1
2. Fix You | Pop | Played: 2
3. Numb | Rock | Played: 1
4. Believer | Rock | Played: 0
5. Perfect | Pop | Played: 1
6. i love you so much | Pop | Played: 0
7. GOD | R&B | Played: 0
8. take me home | Country | Played: 1
9. To the bone | sad | Played: 0
pilih lagu: 5

now playing: Perfect

===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
9

rekomendasi
- Yellow
- Fix You
- i love you so much

```

Jika kita habis memutar sebuah lagu dan memilih menu rekomendasi maka rekomendasi akan terisi dengan genre yang sama dari lagu yang habis kita putar.

10.

```

===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
10
input genre: Rock

Genre: Rock
- Numb | Played: 1
- Believer | Played: 0

```

ini adalah fitur pencarian genre dimana user bisa mencari genre yang dia mau dan nanti akan muncul lagu lagu yang bergenre itu sebagai list.

11.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
0
(base) PS D:\myjavaproject\mysendiri>
```

Jika kita memilih keluar maka program akan otomatis berhenti.

12.

```
===== MUSIC PLAYER =====
1. tambah lagu
2. tampil lagu
3. putar lagu
4. bikin playlist
5. putar playlist
6. history
7. cari lagu
8. top song
9. rekomendasi
10. cari genre
0. keluar
pilih:
12
pilihan gak ada!
```

sistem anti eror jika user memasukan pilihan yang tidak ada.

Tabel penggunaan struktur data :

No.	MATERI	POIN	PERAN DALAM PROGRAM
1	List	5	Digunakan untuk menyimpan seluruh data lagu yang ada di music player agar mudah ditampilkan dan diakses berdasarkan index.
2	Stack	5	Digunakan untuk menyimpan history lagu yang terakhir diputar menggunakan konsep LIFO (Last In First Out).
3	Queue	5	Digunakan untuk membuat playlist lagu menggunakan konsep FIFO (First In First Out). Lagu yang pertama masuk playlist akan diputar lebih dulu.
4	Set	5	Digunakan untuk mencegah judul lagu yang sama agar tidak tersimpan dua kali di dalam program.
5	Map	5	Digunakan untuk mempercepat pencarian lagu berdasarkan judul dan membantu sistem rekomendasi genre lagu.
6	Tree	15	Digunakan untuk mengelompokkan lagu berdasarkan genre dalam bentuk tree genre sehingga lagu dapat dicari berdasarkan kategori genre tertentu.

7	Binary Tree	10	Digunakan dalam bentuk Binary Search Tree (BST) untuk mengurutkan lagu berdasarkan jumlah play agar lagu paling populer lebih mudah dicari.
Total : 50			

Kesimpulan :

Pembuatan program music player ini menunjukkan bahwa penggunaan struktur data yang tepat dapat mempermudah pengelolaan data lagu dan fitur-fitur di dalam program. Struktur data seperti List, Stack, Queue, Map, Tree, dan Binary Search Tree membantu program berjalan lebih teratur, efisien, dan mudah dikembangkan.

Catatan: poin dihitung berdasarkan jenis struktur data, bukan banyaknya struktur data. Dengan demikian, apabila program memiliki list sebanyak 3, maka tetap dihitung 5 poin bukan 15 poin. **Tidak ada toleransi keterlambatan dalam pengumpulan responsi.**

Penilaian Laporan Responsi:

- Nilai kelengkapan laporan antara 0-20.
- Nilai kerapian antara 0-10.
- Nilai dari analisis kompleksitas adalah 0-20.

Total Kode + Laporan = 100